Lab Assignment 2

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as mp
import seaborn as sms
```

In [2]:
```
df=pd.read_csv("academic_performance.csv")
```

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Student_ID  30 non-null     int64
 1   Maths       28 non-null     float64
 2   Science     28 non-null     float64
 3   English     28 non-null     float64
 4   Attendance  28 non-null     float64
dtypes: float64(4), int64(1)
memory usage: 1.3 KB
```

In [4]:
```
df
```

Out[4]:

	Student_ID	Maths	Science	English	Attendance
0	1	78.0	82.0	75.0	85.0
1	2	85.0	79.0	80.0	90.0
2	3	NaN	88.0	70.0	78.0
3	4	95.0	NaN	85.0	95.0
4	5	40.0	35.0	NaN	60.0
5	6	88.0	90.0	88.0	NaN
6	7	150.0	85.0	65.0	88.0
7	8	72.0	400.0	78.0	92.0
8	9	66.0	70.0	68.0	75.0
9	10	55.0	60.0	58.0	65.0
10	11	90.0	92.0	94.0	98.0
11	12	48.0	52.0	50.0	55.0
12	13	82.0	78.0	80.0	82.0
13	14	76.0	NaN	74.0	80.0
14	15	NaN	65.0	60.0	70.0
15	16	68.0	72.0	70.0	77.0
16	17	85.0	88.0	90.0	89.0
17	18	92.0	95.0	98.0	105.0
18	19	35.0	30.0	40.0	45.0
19	20	60.0	58.0	62.0	68.0
20	21	110.0	85.0	88.0	90.0
21	22	73.0	75.0	72.0	NaN
22	23	80.0	82.0	78.0	85.0
23	24	65.0	67.0	70.0	72.0
24	25	90.0	88.0	NaN	95.0
25	26	55.0	50.0	48.0	60.0
26	27	78.0	76.0	80.0	83.0
27	28	82.0	85.0	88.0	87.0
28	29	95.0	98.0	96.0	99.0
29	30	20.0	25.0	30.0	35.0

FOR FINDING INCONSISTENT VALUE IN EVERY COLUMN

In [5]:
```
for col in ['Maths','Science','English','Attendance']:
    print("Inconsistent data in ",col)
    print(df[(df[col]<0) | (df[col]>100)])
```

```
Inconsistent data in  Maths
    Student_ID  Maths  Science  English  Attendance
6            7  150.0     85.0     65.0        88.0
20          21  110.0     85.0     88.0        90.0
Inconsistent data in  Science
   Student_ID  Maths  Science  English  Attendance
7           8   72.0    400.0     78.0        92.0
Inconsistent data in  English
Empty DataFrame
Columns: [Student_ID, Maths, Science, English, Attendance]
Index: []
Inconsistent data in  Attendance
    Student_ID  Maths  Science  English  Attendance
17          18   92.0     95.0     98.0       105.0
```

LABEL BASED SLICING (LOC) TO CONVERT INCONSISTENT DATA TO NAN

In [7]:
```
for col in ['Maths','Science','English','Attendance']:
  df.loc[(df[col] < 0) | (df[col] > 100), col] = np.nan
```

In [8]:
```
df
```

Out[8]:

	Student_ID	Maths	Science	English	Attendance
0	1	78.0	82.0	75.0	85.0
1	2	85.0	79.0	80.0	90.0
2	3	NaN	88.0	70.0	78.0
3	4	95.0	NaN	85.0	95.0
4	5	40.0	35.0	NaN	60.0
5	6	88.0	90.0	88.0	NaN
6	7	NaN	85.0	65.0	88.0
7	8	72.0	NaN	78.0	92.0
8	9	66.0	70.0	68.0	75.0
9	10	55.0	60.0	58.0	65.0
10	11	90.0	92.0	94.0	98.0
11	12	48.0	52.0	50.0	55.0
12	13	82.0	78.0	80.0	82.0
13	14	76.0	NaN	74.0	80.0
14	15	NaN	65.0	60.0	70.0
15	16	68.0	72.0	70.0	77.0
16	17	85.0	88.0	90.0	89.0
17	18	92.0	95.0	98.0	NaN
18	19	35.0	30.0	40.0	45.0
19	20	60.0	58.0	62.0	68.0
20	21	NaN	85.0	88.0	90.0
21	22	73.0	75.0	72.0	NaN
22	23	80.0	82.0	78.0	85.0
23	24	65.0	67.0	70.0	72.0
24	25	90.0	88.0	NaN	95.0
25	26	55.0	50.0	48.0	60.0
26	27	78.0	76.0	80.0	83.0
27	28	82.0	85.0	88.0	87.0
28	29	95.0	98.0	96.0	99.0
29	30	20.0	25.0	30.0	35.0

In [9]:
```
df.isnull().sum()
```

Out[9]:
```
Student_ID    0
Maths         4
Science       3
English       2
Attendance    3
dtype: int64
```

In [10]:
```
#df.fillna(0)
#df.replace(np.nan,0)
df.mean()
```

Out[10]:
```
Student_ID    15.500000
Maths         71.269231
Science       72.222222
English       72.678571
Attendance    77.703704
dtype: float64
```

In [18]:
```
# df["Maths"]=df["Maths"].fillna(df["Maths"].mean())
# df["Science"]=df["Science"].fillna(df["Science"].mean())
# df["English"]=df["English"].fillna(df["English"].mean())
# df["Attendance"]=df["Attendance"].fillna(df["Attendance"].mean())
for col in ['Maths','Science','English','Attendance']:
  df[col]=df[col].fillna(df[col].mean())
```

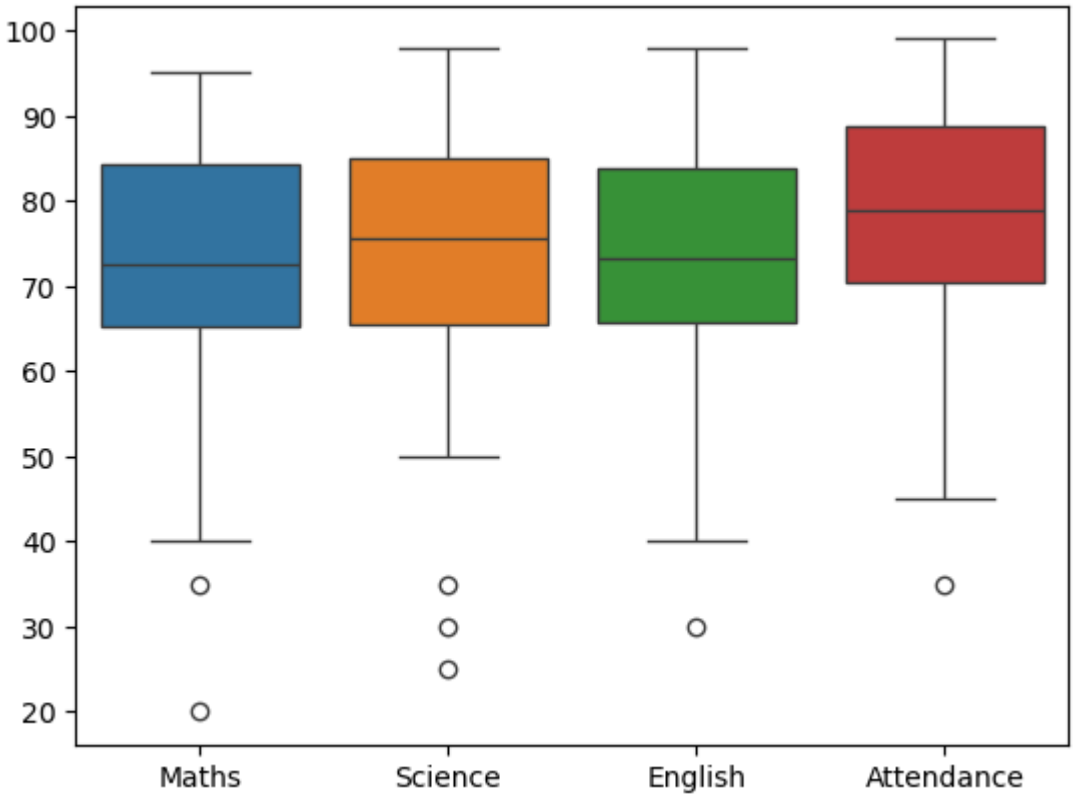
In [16]:
```
df
```

Out[16]:

	Student_ID	Maths	Science	English	Attendance
0	1	78.000000	82.000000	75.000000	85.000000
1	2	85.000000	79.000000	80.000000	90.000000
2	3	71.269231	88.000000	70.000000	78.000000
3	4	95.000000	72.222222	85.000000	95.000000
4	5	40.000000	35.000000	72.678571	60.000000
5	6	88.000000	90.000000	88.000000	77.703704
6	7	71.269231	85.000000	65.000000	88.000000
7	8	72.000000	72.222222	78.000000	92.000000
8	9	66.000000	70.000000	68.000000	75.000000
9	10	55.000000	60.000000	58.000000	65.000000
10	11	90.000000	92.000000	94.000000	98.000000
11	12	48.000000	52.000000	50.000000	55.000000
12	13	82.000000	78.000000	80.000000	82.000000
13	14	76.000000	72.222222	74.000000	80.000000
14	15	71.269231	65.000000	60.000000	70.000000
15	16	68.000000	72.000000	70.000000	77.000000
16	17	85.000000	88.000000	90.000000	89.000000
17	18	92.000000	95.000000	98.000000	77.703704
18	19	35.000000	30.000000	40.000000	45.000000
19	20	60.000000	58.000000	62.000000	68.000000
20	21	71.269231	85.000000	88.000000	90.000000
21	22	73.000000	75.000000	72.000000	77.703704
22	23	80.000000	82.000000	78.000000	85.000000
23	24	65.000000	67.000000	70.000000	72.000000
24	25	90.000000	88.000000	72.678571	95.000000
25	26	55.000000	50.000000	48.000000	60.000000
26	27	78.000000	76.000000	80.000000	83.000000
27	28	82.000000	85.000000	88.000000	87.000000
28	29	95.000000	98.000000	96.000000	99.000000
29	30	20.000000	25.000000	30.000000	35.000000

In [19]:
```
sms.boxplot(data=df[['Maths','Science','English','Attendance']])
```

Out[19]: <Axes: >



DETECT THE OUTLIERS USING BOXPLOT and USE QUANTILE METHOD

FOR Q1=0.25 Q2=0.50 Q3=0.75

In [20]:
```
cols=['Maths','Science','English','Attendance']
```

In [21]:
```
all_Q1=df[cols].quantile(0.25)
```

In [22]:
```
all_Q1
```

Out[22]:
```
Maths         65.25
Science       65.50
English       65.75
Attendance    70.50
Name: 0.25, dtype: float64
```

In [23]:
```
all_Q2=df[cols].quantile(0.50)
```

In [24]:
```
all_Q2
```

Out[24]:
```
Maths         72.500000
Science       75.500000
English       73.339286
Attendance    79.000000
Name: 0.5, dtype: float64
```

In [25]:
```
all_Q3=df[cols].quantile(0.75)
```

In [26]:
```
all_Q3
```

Out[26]:
```
Maths         84.25
Science       85.00
English       83.75
Attendance    88.75
Name: 0.75, dtype: float64
```

In [27]:
```
IQR=all_Q3-all_Q1
lower=all_Q1-1.5*IQR
print("IQR",IQR)
print("LOW",lower)
upper=all_Q3+1.5*IQR
print("upper",upper)
```

```
IQR Maths         19.00
Science       19.50
English       18.00
Attendance    18.25
dtype: float64
LOW Maths         36.750
Science       36.250
English       38.750
Attendance    43.125
dtype: float64
upper Maths         112.750
Science       114.250
English       110.750
Attendance    116.125
dtype: float64
```

In [28]:
```
for col in cols:
 df.loc[(df[col] < lower[col]) , col] = lower[col]
 df.loc[(df[col] > upper[col]) , col] = upper[col]
```

In [29]:
```
df
```

Out[29]:

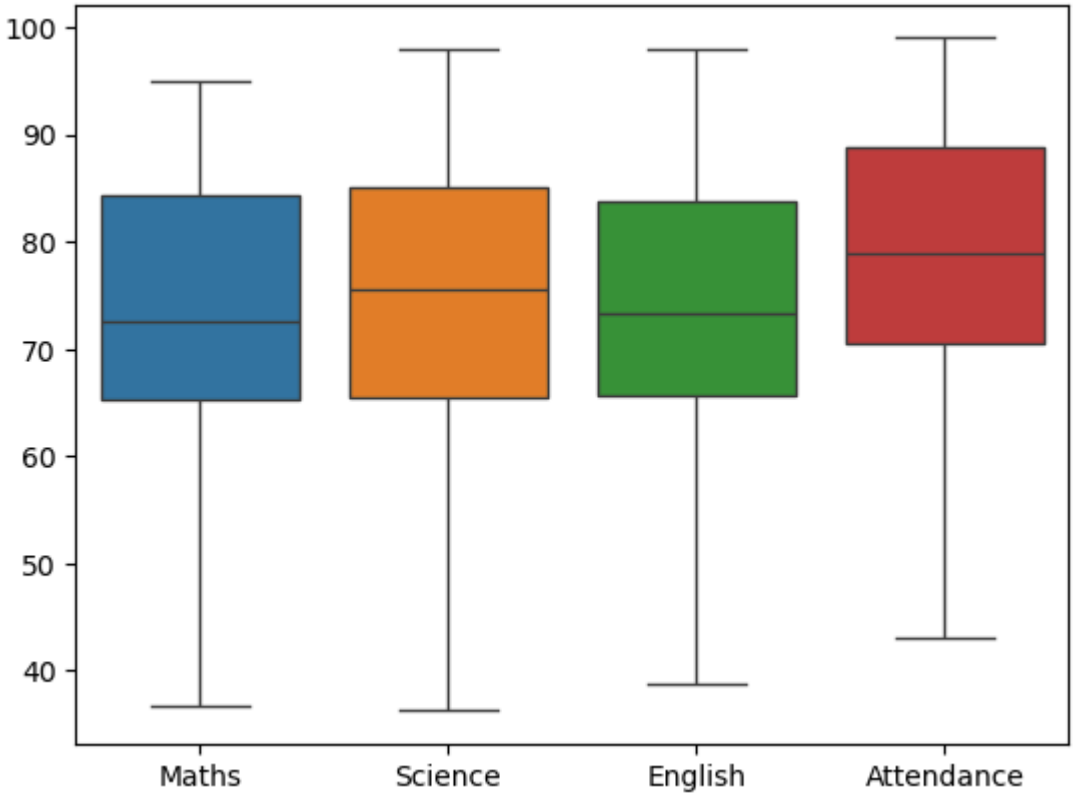
	Student_ID	Maths	Science	English	Attendance
0	1	78.000000	82.000000	75.000000	85.000000
1	2	85.000000	79.000000	80.000000	90.000000
2	3	71.269231	88.000000	70.000000	78.000000
3	4	95.000000	72.222222	85.000000	95.000000
4	5	40.000000	36.250000	72.678571	60.000000
5	6	88.000000	90.000000	88.000000	77.703704
6	7	71.269231	85.000000	65.000000	88.000000
7	8	72.000000	72.222222	78.000000	92.000000
8	9	66.000000	70.000000	68.000000	75.000000
9	10	55.000000	60.000000	58.000000	65.000000
10	11	90.000000	92.000000	94.000000	98.000000
11	12	48.000000	52.000000	50.000000	55.000000
12	13	82.000000	78.000000	80.000000	82.000000
13	14	76.000000	72.222222	74.000000	80.000000
14	15	71.269231	65.000000	60.000000	70.000000
15	16	68.000000	72.000000	70.000000	77.000000
16	17	85.000000	88.000000	90.000000	89.000000
17	18	92.000000	95.000000	98.000000	77.703704
18	19	36.750000	36.250000	40.000000	45.000000
19	20	60.000000	58.000000	62.000000	68.000000
20	21	71.269231	85.000000	88.000000	90.000000
21	22	73.000000	75.000000	72.000000	77.703704
22	23	80.000000	82.000000	78.000000	85.000000
23	24	65.000000	67.000000	70.000000	72.000000
24	25	90.000000	88.000000	72.678571	95.000000
25	26	55.000000	50.000000	48.000000	60.000000
26	27	78.000000	76.000000	80.000000	83.000000
27	28	82.000000	85.000000	88.000000	87.000000
28	29	95.000000	98.000000	96.000000	99.000000
29	30	36.750000	36.250000	38.750000	43.125000

In [30]:
```
df.median()
```

Out[30]:
```
Student_ID    15.500000
Maths         72.500000
Science       75.500000
English       73.339286
Attendance    79.000000
dtype: float64
```

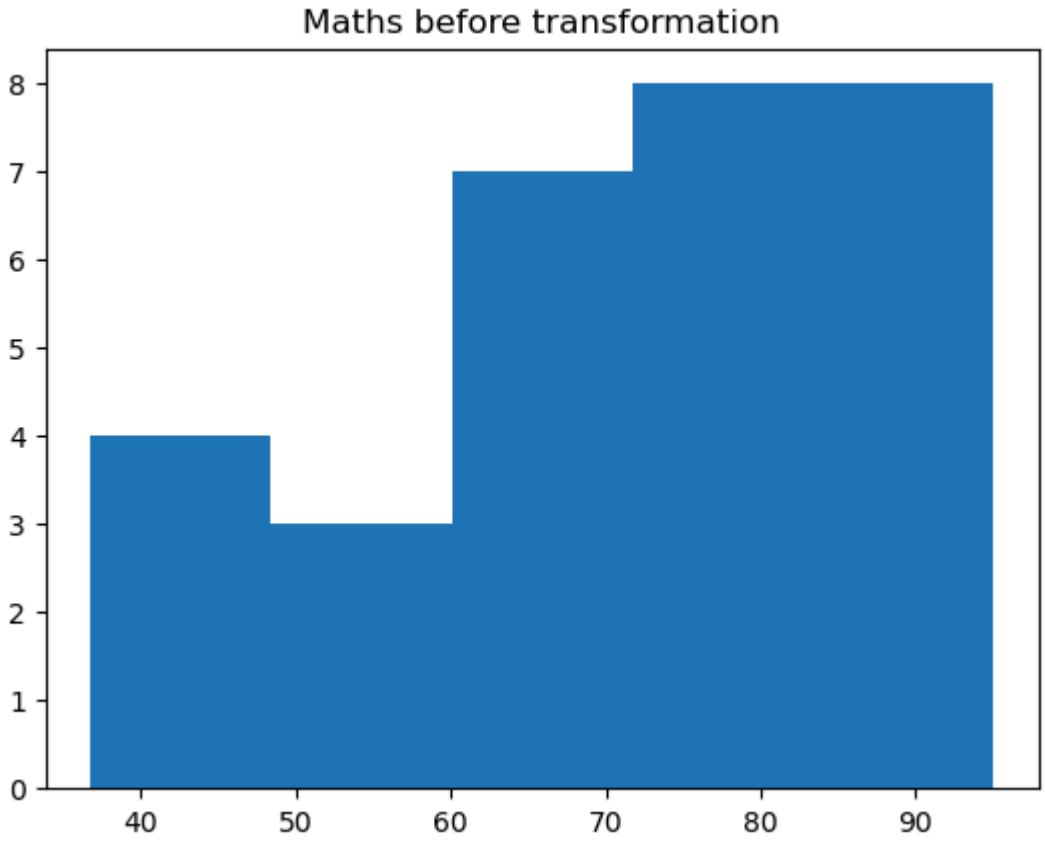
In [31]:
```
sms.boxplot(data=df[['Maths','Science','English','Attendance']])
```

Out[31]:
```
<Axes: >
```



TRANSFORMATION range -0.5 to 0.5 approximately symmetric(+ve) -1 < -0.5 and 0.5 < 1 moderately symmetric(ve)

In [32]:
```
mp.hist(df['Maths'],bins=5)
mp.title("Maths before transformation")
mp.show()
```



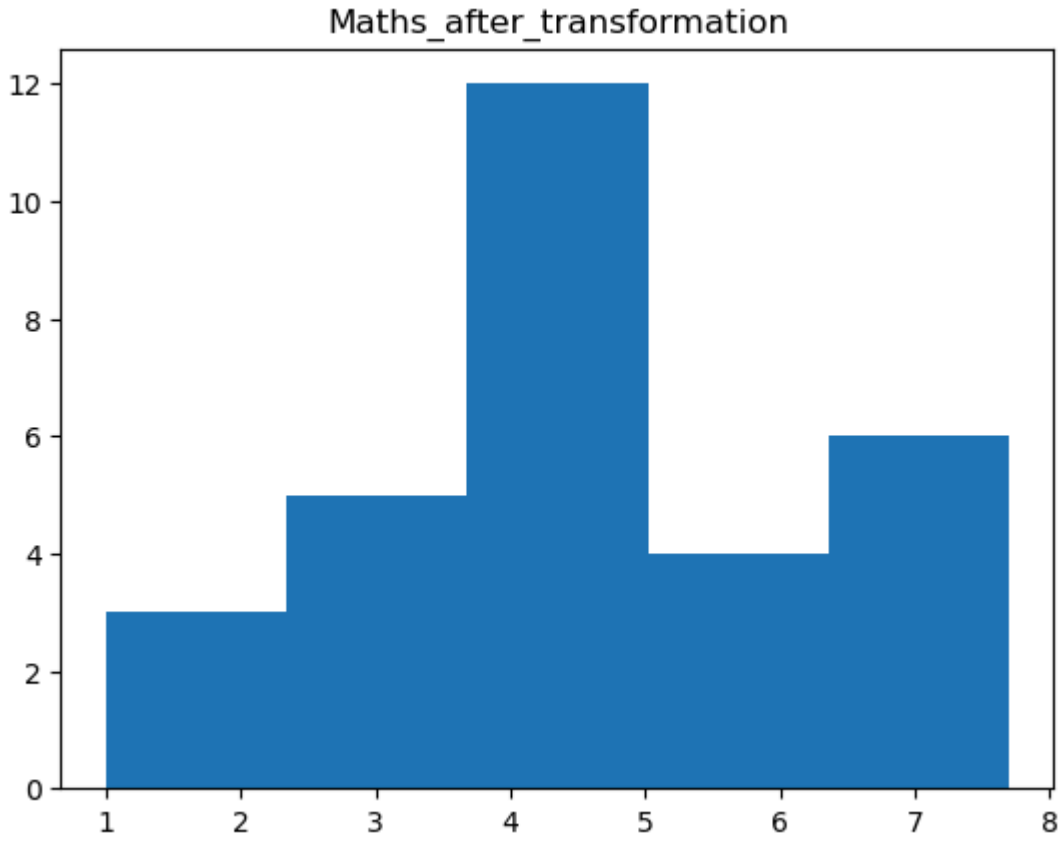
In [35]:
```
df["Maths"].skew()
```

Out[35]:
```
np.float64(-0.7157680000711928)
```

In [36]:
```
max=df["Maths"].max()
df["Maths_reflected"]=max+1-df["Maths"]
df["Maths_transformed"]=np.sqrt(df["Maths_reflected"])
df["Maths_transformed"].skew()
```

Out[36]:
```
np.float64(-0.14535539173782094)
```

In [38]:
```
mp.hist(df['Maths_transformed'],bins=5)
mp.title("Maths_after_transformation")
mp.show()
```


Maths_after_transformation
0
2
4
6
8
10
12
1
2
3
4
5
6
7
8